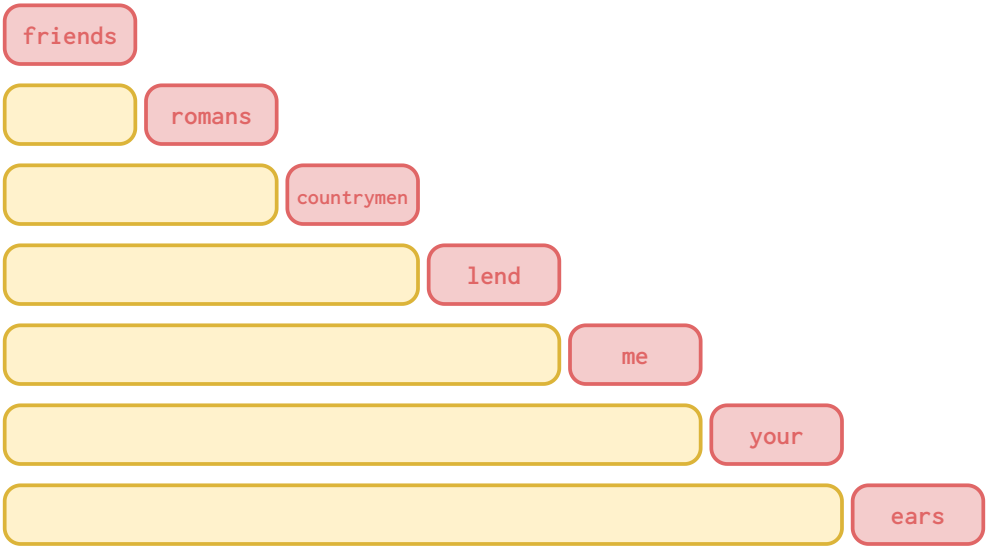


UNIT 01 · CHAPTER 7

Recurrent Neural Networks

CSE 10124 · Building ChatGPT
William Theisen · University of Notre Dame

Next-Token Prediction

Every language model we'll build in this course, irishQwen included, does one thing over and over: **it reads some text and guesses the token that comes next**. A chatbot's answer is just that guess made a few hundred times in a row, each new token stuck onto the end of the text before the next guess is made.

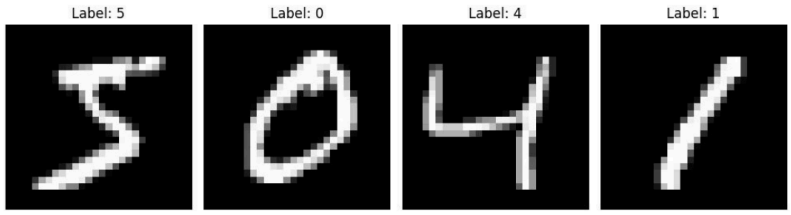
Our text for this chapter comes from the class's Shakespeare corpus: Mark Antony's speech at Caesar's funeral in *Julius Caesar*, the one that opens "Friends, Romans, countrymen, lend me your ears". Given the prompt "lend me your", we'd like a model to say "ears".

To keep the tables small we'll split the speech into words, as the slides do, rather than into irishQwen's BPE tokens. The speech (up to Antony pausing, "till it come back to me") is **268** words long and uses **135** different words, so our vocabulary has **135** tokens.

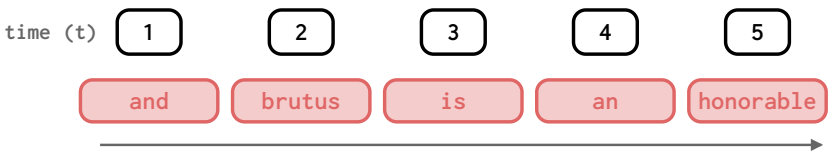
In the last chapter every token became a vector, its embedding, and we saw that a pile of embeddings can't tell which word came first. Unfortunately **the right next word almost never depends on just the last word**, so the rest of this chapter is about a network that reads the embeddings in order and keeps track of the words before.

Reading vs. Seeing

When we see something, we generally take it in all at once. A digit classifier gets all **784** pixels of an image together, and it doesn't really matter which image it saw before: a 5 followed by a 0 is still a 5 and a 0.



Reading is different. We read in a sequence, one word at a time, so **reading has a dimension of time**, which we'll call `t`. Pictured below is a prompt from the speech, "and brutus is an honorable", with one token at each time step.



Notice that a prompt doesn't have a fixed size either. An image classifier always gets **784** pixels, while a prompt can be one word or a whole play.

Context

As we read across time, what we've seen before matters a lot. It provides the **context** for the token we're reading now. Take the word "honorable", which turns up five times in the speech:

```
import re
from collections import Counter

text = open("shakespeare.txt").read()
speech = text[text.find("Friends, Romans"):]
speech = speech[:speech.find("come back to me.") + 16]
words = re.findall(r"[a-z']+", speech.lower())
print(len(words), "words,", len(set(words)), "different")

pairs = list(zip(words, words[1:]))
print(Counter(b for a, b in pairs if a == "honorable"))
print(Counter(b for a, b in pairs if a == "was"))
```

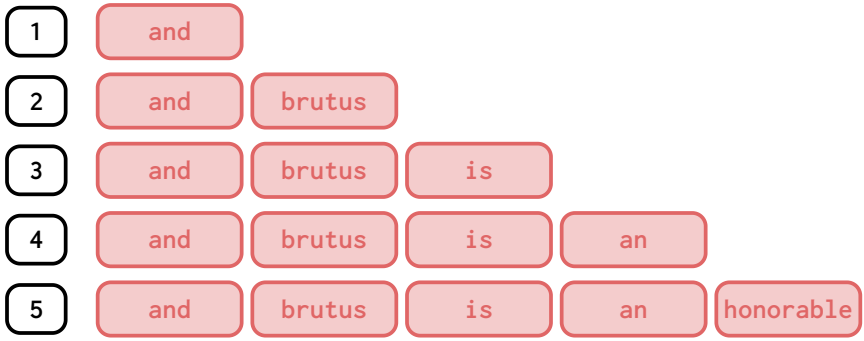
OUTPUT

```
268 words, 135 different
Counter({'man': 4, 'men': 1})
Counter({'ambitious': 4, 'a': 1, 'my': 1, 'this': 1})
```

Four times the next word is "man" ("Brutus is an honorable man") and once it's "men" ("So are they all, all honorable men"). Knowing only the last word, the best we could do is say "man" four times out of five. **The word that decides it is further back**: "Brutus" is one man, "they" are several. "was" is the same story, followed by "ambitious" four times out of seven, but every time "Brutus says he was" comes before it, the next word is "ambitious".

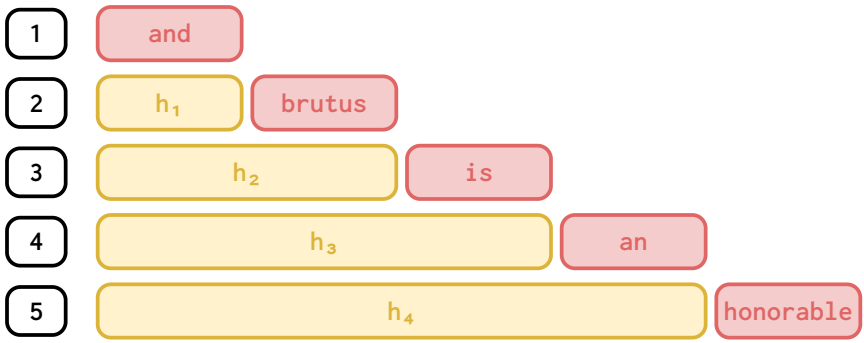
Available Context

So given a prompt, our goal is to predict the next token using everything that came before it. At time step **1** the only context we have is "and". By step **5** it's all five tokens. Pictured below is the context available at each step, growing by one token every time.



A Running Summary

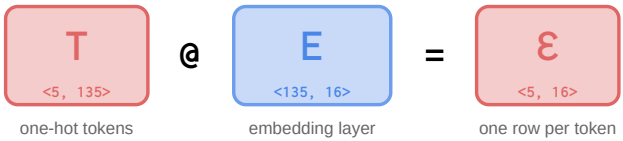
We could try handing a model every token so far, but then it would need a different number of inputs at every step (and a real prompt can be thousands of tokens long). Instead we'll do what a reader does: **keep a running summary of what we've read so far**, call it h, and update it as each new token arrives.



h_1 sums up "and", h_2 sums up "and brutus", and so on. At step **5** the model only needs two things, h_4 and the new token "honorable". h stays the same size however long the prompt gets.

Embeddings for a Prompt

Before anything can be summed up, the prompt has to become numbers. As in Text Embeddings, each token ID becomes a one-hot row, and the one-hot rows T times the embedding layer E give one embedding per token:




```python
import re, torch
text = open("shakespeare.txt").read()
speech = text[text.find("Friends, Romans"):]
speech = speech[:speech.find("come back to me.") + 16]
words = re.findall(r"[a-z']+", speech.lower())
vocab = sorted(set(words))
prompt = "and brutus is an honorable".split()
ids = torch.tensor([vocab.index(w) for w in prompt])
T = torch.nn.functional.one_hot(ids, len(vocab)).float()
torch.manual_seed(0)
E = torch.randn(len(vocab), 16)   # the embedding layer
emb = T @ E                       # one row per token
print(ids)
print(T.shape, E.shape, emb.shape)
```

OUTPUT

```
tensor([ 7, 18, 60,  6, 55])
torch.Size([5, 135]) torch.Size([135, 16]) torch.Size([5, 16])
```

"and" is token **7**, "brutus" token **18** (the vocabulary is in alphabetical order). We'll write $Ɛ_t$ for the row of Ɛ that belongs to the token at time t. Our embeddings have **16** numbers (the slides use **768**), and E is random since nothing is trained yet.

Remembering Context

As we read through our prompt over time, we need to remember, maintain and build up the context of the embeddings we've seen so far. A **recurrent neural network** (RNN) does this with one equation, used at every time step:

[7.1] the context after reading token t

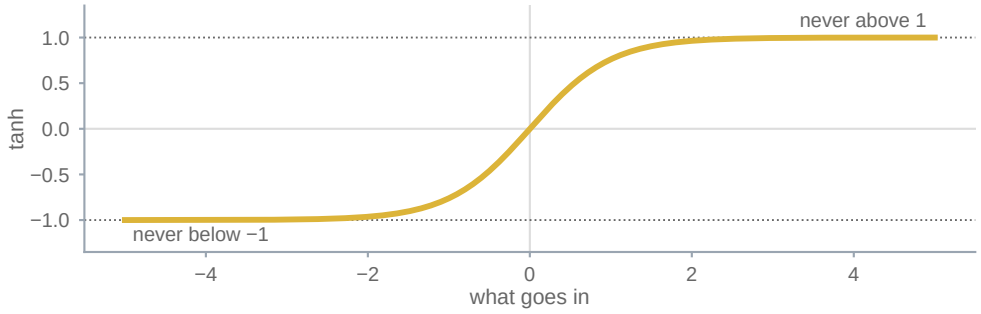
$$h_t = \tanh(h_{t-1} W_h + \mathcal{E}_t W_x + b_h)$$

h_{t-1} is the context from the step before, $\mathcal{E}_t$ the embedding of the token we're reading now, and h_t the new context. The old context and the new token each get multiplied by their own weight matrix, W_h and W_x, a bias b_h is added, and tanh squashes the result. Before we've read anything there is no context, so h_0 is all zeros.

It's called **recurrent** because h_t is fed back in to make h_{t+1}. Essentially **the network's output at one step becomes part of its input at the next**. The idea goes back to the **1980**s, and Jeffrey Elman's **1990** "simple recurrent network" is the version most people mean by an RNN.

The Tanh Function

The **tanh** (hyperbolic tangent) squashes whatever goes in to somewhere between **−1** and **1**. Big positive numbers come out close to **1**, big negative numbers close to **−1**, and numbers near **0** come out nearly unchanged.



Why bother? h gets fed back in at every step. Without the squash, a context that grew a little at each step would keep on growing, and a few hundred tokens later its numbers would be enormous. With tanh **every number in h stays between −1 and 1**, however many tokens we read.

Context by Hand

Here's an RNN small enough to work out by hand. The embeddings and the context have only **2** numbers each, and we picked the weights ourselves so the arithmetic stays simple:

the three embeddings

$$\varepsilon_{\text{lend}} = \begin{bmatrix} 1 & 0 \end{bmatrix} \qquad \varepsilon_{\text{me}} = \begin{bmatrix} 0 & 1 \end{bmatrix} \qquad \varepsilon_{\text{your}} = \begin{bmatrix} 1 & 1 \end{bmatrix}$$

the weights

$$w_x = \begin{bmatrix} 1 & -1 \\ 0.5 & 1 \end{bmatrix} \qquad w_h = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \qquad b_h = \begin{bmatrix} 0 & 0 \end{bmatrix}$$

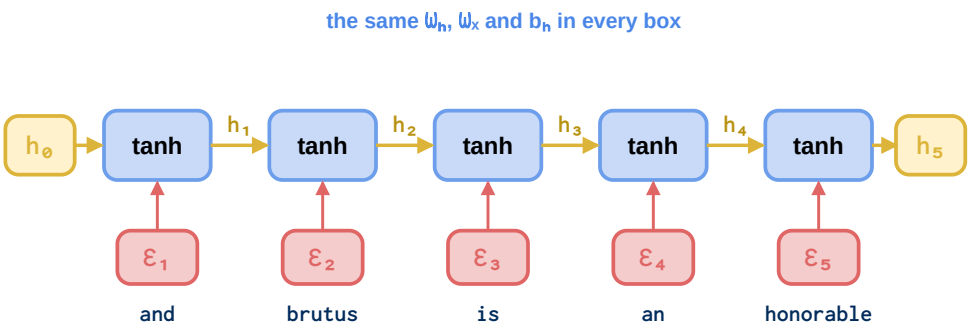
At step **1** the old context h_0 is all zeros, so only the token counts: $h_1 = \tanh([\mathbf{1}, \mathbf{-1}]) = [\mathbf{0.762}, \mathbf{-0.762}]$. From step **2** on, the old context joins in:

t	token	$h_{t-1}\, w_h$	$\varepsilon_t\, w_x$	sum	h_t
1	lend	[0, 0]	[1, −1]	[1, −1]	[0.762, −0.762]
2	me	[0.381, −0.381]	[0.5, 1]	[0.881, 0.619]	[0.707, 0.551]
3	your	[0.353, 0.275]	[1.5, 0]	[1.853, 0.275]	[0.952, 0.269]

Now read just "me your". The last token is the same, but h ends at [**0.939**, **0.363**] instead of [**0.952**, **0.269**], because this time it never saw "lend". **The context after "your" still carries a trace of "lend"**, two steps back.

Unrolling

Because the context loops back into the same equation, an RNN is usually drawn **unrolled**, with one box per time step, side by side. Pictured below is our prompt.



It's important to remember that these aren't five different layers. **Every box uses the same weights**, W_h, W_x and b_h, so an RNN that reads `5` tokens has exactly as many weights as one that reads `5000`. That's how it copes with prompts of any length.

Notice also that the only thing passed from one box to the next is h. Whatever the RNN remembers about "and brutus is an" has to be in h_4.

Building Context in Code

In PyTorch the whole RNN is a loop over the prompt (`Wx` is W_x and `Wh` is W_h, since W is hard to type). Our context has **64** numbers:

```
import re, torch
text = open("shakespeare.txt").read()
speech = text[text.find("Friends, Romans"):]
speech = speech[:speech.find("come back to me.") + 16]
words = re.findall(r"[a-z']+", speech.lower())
vocab = sorted(set(words))
prompt = "and brutus is an honorable".split()

torch.manual_seed(0)
E = torch.randn(len(vocab), 16)   # embeddings
Wx = torch.randn(16, 64) * 0.1    # weights for the token
Wh = torch.randn(64, 64) * 0.1    # weights for context
bh = torch.zeros(64)

h = torch.zeros(64)               # h_0: no context yet
for t, word in enumerate(prompt):
    emb = E[vocab.index(word)]
    h = torch.tanh(h @ Wh + emb @ Wx + bh)
    print(t + 1, word.ljust(9), h[:4])
print(h.shape)
```

OUTPUT

```
1 and       tensor([-0.0462,  0.2937,  0.4460,  0.1700])
2 brutus    tensor([-0.3250, -0.5522,  0.2123, -0.7765])
3 is        tensor([-0.3094, -0.0478,  0.3018,  0.5108])
4 an        tensor([ 0.6320, -0.8885,  0.8699,  0.3384])
5 honorable tensor([ 0.4739, -0.4578,  0.3444, -0.1682])
torch.Size([64])
```

Each line shows the first four of the **64** numbers in h_t. They change at every step and stay between **−1** and **1**. With random weights they don't mean anything yet, but h_5 depends on all five tokens.

Producing Output

After the last token, h_5 should hold everything we need to guess the next word. To turn it into a guess we need one score per word in the vocabulary, so we multiply by one more weight matrix W_y ($\mathbf{64} \times \mathbf{135}$), add a bias b_y, and use a softmax to turn the $\mathbf{135}$ scores into probabilities:

[7.2] one probability per token in the vocabulary

$$\hat{y}_t = \mathrm{softmax}(h_t W_y + b_y)$$

$\hat{y}_t$ is the RNN's guess at the token after the one it just read. Each of the $\mathbf{135}$ columns of W_y belongs to one word, so a word scores highly when h_t lines up with its column. Like W_h and W_x, W_y starts out random and is learned.

Output in Code

Here's our prompt again, with the output layer on the end:

```python
import re, torch
text = open("shakespeare.txt").read()
speech = text[text.find("Friends, Romans"):]
speech = speech[:speech.find("come back to me.") + 16]
words = re.findall(r"[a-z']+", speech.lower())
vocab = sorted(set(words))
torch.manual_seed(0)
E = torch.randn(len(vocab), 16)
Wx = torch.randn(16, 64) * 0.1
Wh = torch.randn(64, 64) * 0.1
bh = torch.zeros(64)
Wy = torch.randn(64, len(vocab)) * 0.1  # h -> vocab
by = torch.zeros(len(vocab))
h = torch.zeros(64)
for word in "and brutus is an honorable".split():
    emb = E[vocab.index(word)]
    h = torch.tanh(h @ Wh + emb @ Wx + bh)
y_hat = torch.softmax(h @ Wy + by, dim=0)
print(y_hat.shape, y_hat.sum())
for p, i in zip(*y_hat.topk(3)):
    print(vocab[i].ljust(7), round(p.item(), 4))
man = y_hat[vocab.index("man")]
print("man".ljust(7), round(man.item(), 4))
```

OUTPUT

```
torch.Size([135]) tensor(1.)
thrice  0.0136
bones   0.0127
and     0.0124
man     0.0064
```

The `135` probabilities add up to `1`. With random weights the RNN's favorite is "thrice" (`0.0136`), and it gives "man" `0.0064`, a little less than the `1/135` = `0.0074` it would get from a blind guess.

Softmax

The scores $h_t W_y + b_y$ can be any numbers at all, positive or negative. **Softmax** turns them into probabilities in two steps: raise e to the power of each score (which makes every one positive), then divide each by their total (which makes them add up to `1`):

[7.3]

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

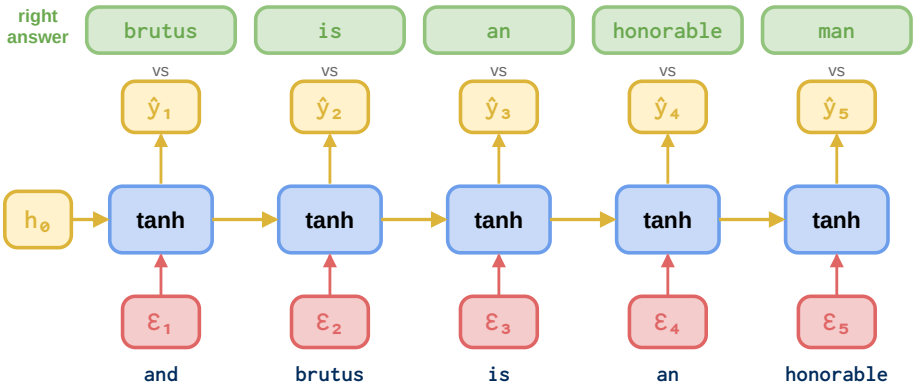
score i out of K

With three words scored `2`, `1` and `0`, the powers are `7.389`, `2.718` and `1`, which add up to `11.107`, so the probabilities are `0.665`, `0.245` and `0.090`. The biggest score gets the biggest probability, and **a score one higher is about `2.7` times as likely**, whatever the other scores are.

Our RNN does the same with `135` scores, one per word, and that list of `135` probabilities is $\hat{y}$.

Output at Every Step

Nothing says we have to wait until the end of the prompt to predict. The same W_y and b_y can turn every h_t into a guess $\hat{y}_t$ at the token after it, and the speech tells us the right answer each time.



So **a five-word prompt is really five predictions**: "brutus" after "and", "is" after "and brutus", all the way to "man" after the whole prompt. When we generate text we only care about the last one, but when we train, every step's guess can be checked against the speech.

Training Samples

To train our RNN we need examples of prompts and the words that follow them, and the speech is full of them. We'll cut it into every run of `8` words in a row. For each run, the targets are the same `8` words shifted along by one, the next word at every step:

```
import re, torch
import torch.nn.functional as F

text = open("shakespeare.txt").read()
speech = text[text.find("Friends, Romans"):]
speech = speech[:speech.find("come back to me.") + 16]
words = re.findall(r"[a-z']+", speech.lower())
vocab = sorted(set(words))
ids = torch.tensor([vocab.index(w) for w in words])

# every 8 words in a row, and the 8 words that follow each
starts = range(len(ids) - 8)
X = torch.stack([ids[i:i + 8] for i in starts])
Y = torch.stack([ids[i + 1:i + 9] for i in starts])
print(X.shape, Y.shape)
```

OUTPUT

```
torch.Size([260, 8]) torch.Size([260, 8])
```

That gives `260` training samples, X, each with `8` targets in Y, so `2080` predictions to check. The first sample is "friends romans countrymen lend me your ears i": reading "friends" it should predict "romans", reading "romans" it should predict "countrymen", and so on until it reads "i" and should predict "come".

Cross-Entropy Loss

To train the RNN we need to **measure how far off each prediction is**. Our prediction is a list of probabilities, one per word, and only one word was right, so all that matters is the probability the RNN gave the right one. The loss for one prediction is minus the log of that probability:

[7.4] p of the right token

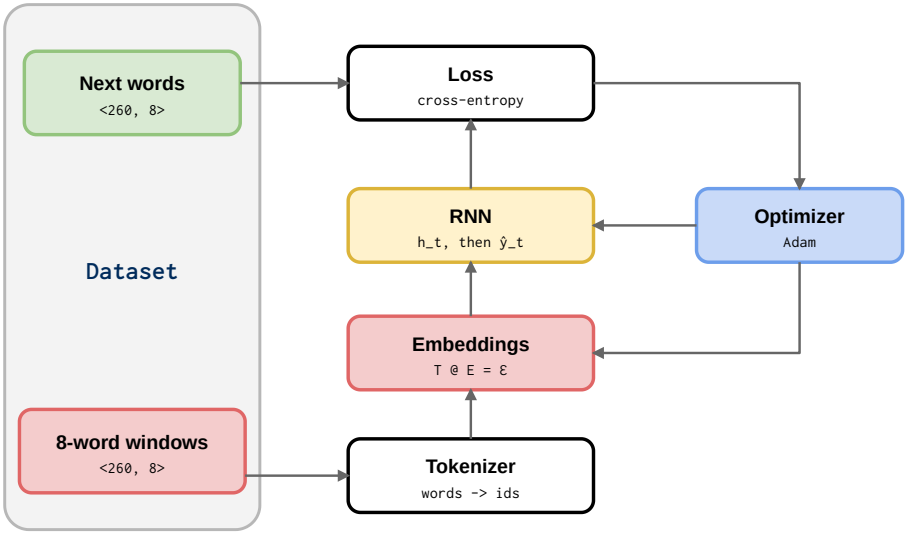
$$L = -\ln \hat{y}_{right}$$

probability of the right word	loss
0.999	0.001
0.785	0.242
0.194	1.640
1/135 = 0.0074 (a blind guess)	4.905

A sure and right prediction costs almost nothing, and the less likely the RNN thought the right word was, the more it costs. This is the **cross-entropy loss**, and for a whole batch we average it over every prediction (`2080` for us). Notice the last row: an RNN that knows nothing should start out near `4.905`.

A Text Prediction Model

Pictured below is everything we've built, put together for training.



The windows go through the tokenizer and the embedding layer, the RNN reads each one a token at a time and predicts at every step, and the loss compares all those predictions with the next words. The optimizer then nudges **every weight, the embeddings included**, in whichever direction lowers the loss, and we go round again.

The one new wrinkle is that W_h is used at every step, so changing it changes all eight predictions at once. Fortunately PyTorch keeps track of every step in the loop for us, and `loss.backward()` works out the gradient through all eight of them.

Training an RNN

Here's the training loop, carrying on from the samples we cut. E is now part of the model and gets trained too, and as in Embeddings for a Prompt the tokens reach it as one-hot rows T:

```
torch.manual_seed(0)
V = len(vocab)
E = torch.randn(V, 16) * 0.1
Wx = torch.randn(16, 64) * 0.1
Wh = torch.randn(64, 64) * 0.1
Wy = torch.randn(64, V) * 0.1
bh, by = torch.zeros(64), torch.zeros(V)
params = [E, Wx, Wh, bh, Wy, by]
for p in params:
    p.requires_grad_()
opt = torch.optim.Adam(params, lr=0.01)
T = F.one_hot(X, V).float()   # one-hot tokens

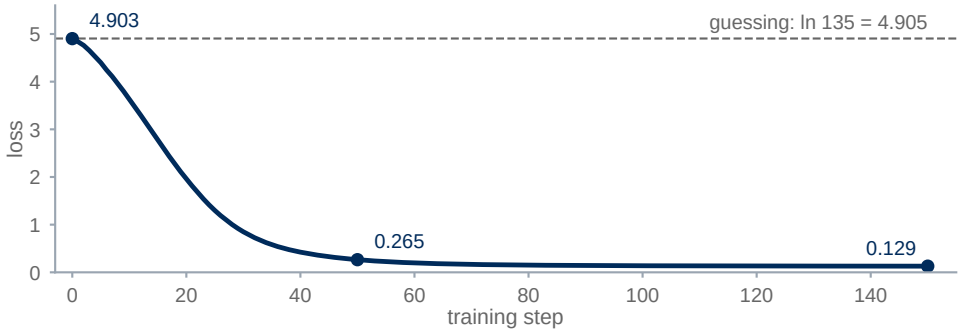
losses = []
for step in range(151):
    h, loss = torch.zeros(len(X), 64), 0
    for t in range(8):
        h = torch.tanh(h @ Wh + T[:, t] @ E @ Wx + bh)
        logits = h @ Wy + by
        loss = loss + F.cross_entropy(logits, Y[:, t])
    loss = loss / 8
    opt.zero_grad()
    loss.backward()
    opt.step()
    losses.append(round(loss.item(), 3))
print("loss at steps 0, 50, 100, 150:", losses[::50])
```

OUTPUT

```
torch.Size([260, 8]) torch.Size([260, 8])
loss at steps 0, 50, 100, 150: [4.903, 0.265, 0.138, 0.129]
```

Watching the Loss

Pictured below is the loss at every one of the **151** steps.



It starts at **4.903**, right where a blind guess would be (**4.905**), and falls to **0.129**. On average the RNN now gives the right next word a probability of about **0.88** ($e^{-0.129}$).

Before getting too excited, our RNN has **16,119** weights (most of them in W_y and the embeddings) and only **2080** predictions to learn. **It has more or less memorized the speech**, which is all we've asked it to do. irishQwen trains on all **5.4** MB of Shakespeare, a much harder test.

What the RNN Learned

Here are the trained RNN's two favorite next words for a few prompts:

```
def context(prompt):
    h = torch.zeros(64)
    for w in prompt:
        emb = E[vocab.index(w)]
        h = torch.tanh(h @ Wh + emb @ Wx + bh)
    return h

def predict(prompt):
    return torch.softmax(context(prompt) @ Wy + by, dim=0)

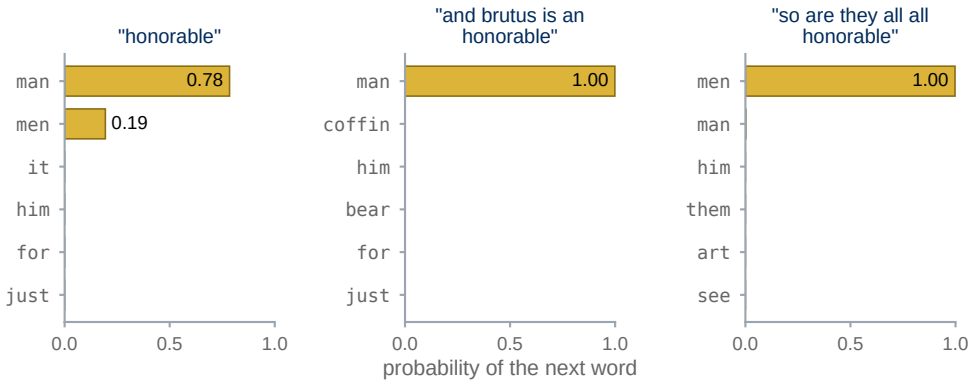
with torch.no_grad():
    for prompt in ["honorable",
                   "and brutus is an honorable",
                   "so are they all all honorable",
                   "was",
                   "brutus says he was",
                   "lend me your"]:
        p, i = predict(prompt.split()).topk(2)
        a, b = p.tolist()
        print(prompt.rjust(29), vocab[i[0]], round(a, 3),
              vocab[i[1]], round(b, 3))
```

OUTPUT

```
torch.Size([260, 8]) torch.Size([260, 8])
loss at steps 0, 50, 100, 150: [4.903, 0.265, 0.138, 0.129]
                    honorable man 0.785 men 0.194
   and brutus is an honorable man 0.999 coffin 0.0
so are they all all honorable men 0.998 man 0.001
                         was ambitious 0.573 my 0.14
           brutus says he was ambitious 0.997 my 0.001
                 lend me your ears 0.997 heart 0.001
```

Context Decides

The interesting rows are the ones that end the same way. Pictured below are the RNN's six favorite next words after "honorable" on its own and after two longer prompts that end in "honorable".



Given just "honorable", the RNN says "man" `0.785` and "men" `0.194`, almost exactly the four out of five and one out of five it saw in the speech. That's the best anything can do with only the last word. Given the whole prompt, it's sure: "man" after "brutus is an honorable" and "men" after "so are they all, all honorable". **Everything that told them apart came through h**, from "brutus" and "they" three steps back. "was" works the same way, `0.573` for "ambitious" on its own and `0.997` after "brutus says he".

Generating Text

We can also let the RNN write on its own. Give it "friends romans", take its favorite next word, stick it on the end of the prompt and ask again, twenty times:

```
text = ["friends", "romans"]
with torch.no_grad():
    for _ in range(20):
        text.append(vocab[predict(text).argmax().item()])
print(" ".join(text[:11]))
print(" ".join(text[11:]))
```

OUTPUT

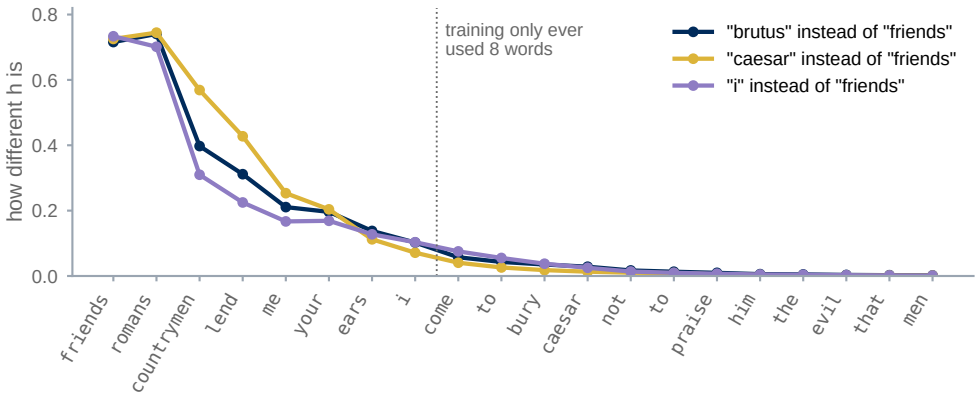
```
torch.Size([260, 8]) torch.Size([260, 8])
loss at steps 0, 50, 100, 150: [4.903, 0.265, 0.138, 0.129]
                    honorable man 0.785 men 0.194
   and brutus is an honorable man 0.999 coffin 0.0
so are they all all honorable men 0.998 man 0.001
                          was ambitious 0.573 my 0.14
           brutus says he was ambitious 0.997 my 0.001
                 lend me your ears 0.997 heart 0.001
friends romans countrymen lend me your ears i come to bury
caesar not to praise him the evil that men do lives
```

The last two lines are word for word the opening of the speech. Of course they are, since **the speech is the only thing it has ever read**. A model that's read more has more than one way to carry on, and always picking the favorite (called greedy decoding) is only one way to choose. That's how a chatbot writes an answer, though: one guess at the next token, added to the end, again and again.

The Bottleneck

Everything the RNN knows about the prompt has to fit through h, `64` numbers for us (`768` on the slides), whether the prompt is five words or five thousand. Each step has to decide what to keep, and every new token pushes a little of the old context out.

We can watch it happen. Below we ran the speech's first `40` words through the trained RNN, then ran them again with only the first word changed, and measured how different h was at every step.



Changing the first word changes h a lot at first (about `0.72`), but by the eighth word the difference is down to about `0.1`, and by the fifteenth it has all but gone. **After a dozen or so words, the RNN has forgotten how the prompt started.** In fairness, ours never trained on more than `8` words at a time, but RNNs trained on long text have the same trouble remembering things from far back.

One Token at a Time

There's a second problem with reading one token at a time. We can't work out h_5 until we have h_4, and we can't work out h_4 until we have h_3. **The steps have to happen in order**, one after another, however many GPUs we have.

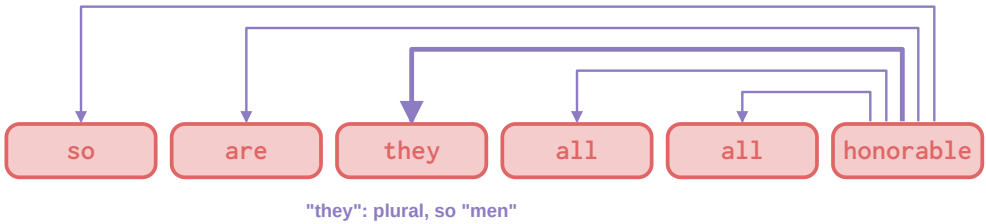
That's the inner loop in our training code, **8** steps that can't be done at the same time. For a prompt of a few thousand tokens it's a few thousand steps in a row, and a GPU is at its best doing thousands of things at once, not one thing thousands of times.

So an RNN has two weaknesses, and they come from the same design choice. Squeezing the past into h loses the start of a long prompt, and building h one step at a time makes it slow.

Looking Back

Think about how the RNN got "men" right after "so are they all, all honorable". The word that mattered, "they", was three steps back, and it only reached the last step by surviving three updates of h. Further back than that, it would probably have been lost.

A more direct approach would be to let the last step look straight back at every earlier token, and decide for itself which ones matter.



Nothing would have to survive a chain of updates, and every token could look back at the same time instead of waiting in line. **That idea is called attention**, and it's where the next chapter, Self-Attention, picks up.